ECE 310 – WIRELESS COMMUNICATION

Milestone 2 (M2) Report

Technical Development and SOTA Positioning

Group 05

Kensi Patel (AU2440102), Maitri Lad (AU2440146), Rishika Bothra (AU2440179), Nikkesh Parekh (AU2440214), Aryan Thakkar (AU2440248)

Title: Open-Set / "Unknown Signal" Aware AI Classifier for GNU Radio

Category: GSoC (Google Summer of Code) – AI Spectrum in GNU Radio

Project Repository: https://github.com/Aryanth29/g5_gsoc_spectrumsensing

Upstream Repository:https://github.com/gnuradio/gnuradio

1. Introduction and Objectives

In M1 we defined the problem (closed-set classifiers cannot flag unknown signals), surveyed the literature, and identified the SOTA gap we intended to fill. In M2 we shifted to hands-on engineering:

- Studied two accepted Google Summer of Code (GSoC) 2026 proposals to understand how successful contributors scope and deliver GNU Radio projects.
- Worked on four open issues in the upstream GNU Radio repository, submitting code fixes, new features, and QA tests.
- Submitted Pull Request #8224 (tcp_sink QA), Pull Request #8230 (Vector Sink marker cleanup), and Pull Request #8232 (bokehgui crash guard) to the upstream repository (all open, under maintainer review).
- Prepared code and a patch for one additional contribution (Issue #5672), ready for PR submission.
- Updated our SOTA positioning by comparing our proposed system against existing tools and research.

Going into M3, based on this hands-on experience, the team is shifting its engineering focus away from the AI open-set classifier pipeline originally scoped in M1, and towards building and integrating real GNU Radio signal-processing infrastructure. Our M3 goals are:

- GRC Blocks learn to create a new block in GRC (GNU Radio Companion) and integrate it into the existing GNU Radio environment.
- Spectrum Sensing creates and adds a Spectrum Sensing block with its signal-processing framework implemented.
- ISAC Framework rebuilds the ISAC (Integrated Sensing and Communication) end-to-end framework into GNU Radio, ensuring the transceiver works end-to-end.

2. GSoC 2026 Proposals Reviewed

In order to gain insight into the structure of successful open source contributions and what GNU Radio mentors care about, we examined in detail two accepted GSoC 2026 projects. There is absolutely no AI or signal classification involved in each project; both are a representation of

engineering discipline and limited deliverables, which is highly valued in the GNU Radio community.

2.1 Project 1: Hardware-in-the-Loop (HIL) CI

Contributor: Sayantan Maity **Mentors:** Cyrille Morin, Marcus Müller

The Problem: GNU Radio has a continuous-integration (CI) pipeline, which is a 100% software simulation. For the same reason, code that works on simulated radios may not work when installed on real radio gear: timing, driver behavior, antenna interference, and other physical-world factors.

What He Built: A controller written in Python which automates the entire process of hardware-testing:

Reserves radio devices at CortexLab (remote lab with rentable SDR nodes) with OAR job scheduler.

Starts a MINUS task (environment setup at CortexLab), statuses the nodes, and copies tests to them with SSH/SFTP.

• Starts transmitter and receiver at the same time with a threading barrier.

• Executes the analysis.py script when the test is recorded with IQ data and generates a pass/fail results.json file.

Uses a live dashboard (flask web server) that refreshes itself every two seconds.

Deliverables and Status:

Component	Status
The API and live dashboard that is provided by Flask.	Done
Automate the reservation, nodes and tasks	Done
Experiment format that can be reused (scripts, analysis, parameters).	Done
Basic tone test (send tone, checking tone)	Applies to real hardware, works end-to-end
Sends a digital message.Sends a digital message (OFDM message test).	Runs, but output file not written — needs debugging.
The Dockerfile contains the instructions necessary to build GNU Radio.	Done, but not connected to CI
This will trigger a hardware test when the CI is activated (full automatic).	Not done (future work)

Key Takeaway: This is an infrastructure project — all the plumbing to make it possible to have GNU Radio test every code change on real radios in the future. Shows that system-integration work (reservation, remote execution, result collection) is considered as important as algorithm development.

2.2 Project 2: CyberEther Interoperability - GPU-Accelerated Visualisation

Contributor: Youssef Mahmoud **Mentors:** Luigi Cruz (CyberEther), Håkon Vågsether (GNU Radio)

The Problem: GNU radio plots (frequency view, waterfall, constellation) are not GPU accelerated. CyberEther is an independent project, with quick, gpu-accelerated plots. The idea for the GSoC was to enable the use of CyberEther sinks graphically in GNU Radio flowgraphs.

What He Built: gr-cyberether, an out-of-tree (OOT) module that brings CyberEther's GPU-accelerated visualisation sinks into GNU Radio.

Block	**What It Does**
cyber_lineplot_sink	Line plot of a signal (time domain / frequency domain)
cyber_waterfall_sink	Scrolling waterfall display
cyber_constellation_sink	Digital signal quality scatter plot of IQ.
cyber_spectrum_analyzer_sink	The spectrum line above a waterfall is divided between two FFT's.
cyber_range	Flowgraph variable slider in control panel to modify the value of a flowgraph variable during run time.

Integration: Users configure GNU Radio Companion's Generate Options to cyberether_gui, insert a sink, and hit Run. There is no need for a Python Snippet block or window handling. One window is shared by multiple plots with a "row, col" hint. It is supported on Linux, macOS and Windows (GNU Radio 3.10 or main).

Key Takeaway: This is a user centric feature — A new suite of GPU accelerated, fast plots. The result of the HIL project is not immediately visible for end users as in the case of the latter.

2.3 What We Learned from Both Proposals

Aspect	**Project 1 (HIL CI)**	**Project 2 (CyberEther)**
Type	Testing infrastructure	Visualisation add-on (OOT)
Main skills	Python, Flask, SSH, Docker, scheduling	C++20, GPU graphics, GNU Radio blocks
Who benefits	GNU Radio developers	GNU Radio end users
Biggest unfinished piece	Full CI hookup, OFDM bug	Upstream merge (parts awaiting review)
Relation to AI/classification	None	None

- Both accepted projects are not research but rather have clearly bounded deliverables – a working controller or a set of blocks.
- Both were suggested in the official ideas list and talked to mentors on the mailing list prior to deadlines.
- Both were focused on infrastructure and integration, instead of novelty in research.
- Future proposals should be presented as an OOT module (including blocks and demo), instead of as a new research method to make it competitive.

3. Open-Source Contributions

During M2, our team worked on four open issues in the upstream GNU Radio repository. Three pull requests (#8224, #8230, #8232) have been submitted and are under maintainer review. One additional contribution (Issue #5672) is prepared as a local branch with a patch ready for PR submission.

3.1 Issue #7697 / PR #8224 — QA Coverage for network.tcp_sink

Field	Detail
Issue	#7697 — tcp_source has no QA (title is a typo; the issue asks for tcp_sink QA)
PR	#8224 — network: add tcp_sink QA coverage
Author	KensiPatel
Branch	fix/tcp-sink-qa
Status	Open — under maintainer review, DCO passed, workflows awaiting approval
Files changed	gr-network/python/network/qa_tcp_sink.py (modified)
PR Link	https://github.com/gnuradio/gnuradio/pull/8224

Problem: Issue #7697, filed by maintainer Marcus Müller, pointed out that network.tcp_sink lacked proper QA tests. The similar tcp_source block already had tests in qa_tcp_source.py, but tcp_sink had only a basic restart test with no data-verification coverage.

Technical Approach:

- Client mode test: tcp_sink initiates a connection to a listening TCP socket. A deterministic 1,000-byte pattern (tuple(i % 256 for i in range(1000))) is sent and the received sequence is compared byte-for-byte.
- Server mode test: tcp_sink listens; the test creates a client connection. A custom GateBlock (derived from gr.sync_block) prevents data emission until the TCP connection is ready, using a threading.Event for synchronisation.
- End-to-end test (sink as server): tcp_sink SERVER → tcp_source CLIENT across separate GNU Radio flowgraphs with exact-byte verification.
- End-to-end test (sink as client): tcp_sink CLIENT → tcp_source SERVER — the reverse arrangement.
- Restart test: The pre-existing restart/reconnect test was preserved unchanged.
- Dynamic ports: get_free_port() binds a temporary socket to port 0, retrieves the OS-assigned port, and closes the socket. This prevents port collisions across parallel test runs.

Test Results:

Test	Purpose	Windows Result
test_client_mode	Sink client sends bytes to test server; exact byte comparison	PASS
test_server_mode	Test client connects to sink server; event-gated byte transmission	PASS
test_e2e_sink_server	tcp_sink SERVER → tcp_source CLIENT, separate flowgraphs	SKIPPED (Windows #7695)
test_e2e_sink_client	tcp_sink CLIENT → tcp_source SERVER, separate flowgraphs	SKIPPED (Windows #7695)
test_restart	Existing restart/reconnect test retained	PASS

The two E2E tests are skipped on Windows because tcp_source uses os.dup() on a socket handle, which does not work with Windows socket file descriptors (separate Issue #7695, not part of this PR).

CI Issues Encountered and Resolved:

- E203 formatting violation: GitHub Actions flagged whitespace before ':' in a slice expression. Corrected from self.data[self.offset : self.offset + n] to self.data[self.offset:self.offset + n].
- Missing DCO sign-off: The original commit lacked a Developer Certificate of Origin sign-off. Fixed with git commit --amend --signoff --no-edit and force-pushed with --force-with-lease.
- Workflow approval gate: Three CI workflows await maintainer approval — this is a standard gate for first-time contributors, not a rejection.

3.2 Issue #5672 — Add Toggleable Canvas Grid Display in GRC

Field	Detail
Issue	#5672 — Add option to display grid in GRC canvas
Branch	fix/5672-grc-grid-display
Commit	21b2ae1
Status	Code complete, patch ready — PR not yet submitted
Files changed	5 modified + 1 new test file (6 total)
Lines added	+43 implementation, +179 tests (15 test cases across 4 classes)
Issue Link	https://github.com/gnuradio/gnuradio/issues/5672

Problem: GNU Radio Companion (GRC) has a snap-to-grid feature where blocks snap to invisible grid lines spaced 8 pixels apart. However, these grid lines were never visible — users had to guess where they were. Issue #5672 requested a toggleable, persistent grid display. Maintainer @skoslowski confirmed the issue and provided file hints for implementation.

Technical Approach:

The solution follows existing GRC conventions exactly, touching 6 files:

- grc/gui/Actions.py — Registered a new TOGGLE_GRID_DISPLAY action with preference_name='grid_display', default=False (grid off by default), following the exact pattern of TOGGLE_SNAP_TO_GRID.
- grc/gui/canvas/colors.py — Added CANVAS_GRID_COLOR as #DDDDDD (light gray), visible against the white background but subtle enough not to compete with blocks.
- grc/gui/DrawingArea.py — Added a _draw_grid(cr, width, height) method that draws vertical and horizontal lines at the CANVAS_GRID_SIZE (8px) interval using Cairo. The method uses cr.save()/cr.restore() to isolate Cairo state and prevent line-width corruption of subsequent block rendering.
- grc/gui/Bars.py — Added the toggle to the View menu immediately after Snap to Grid (both are grid-related toggles).
- grc/gui/Application.py — Added preference loading at startup and a toggle handler that redraws all open pages (not just the current tab).

Bugs Found and Fixed During Development:

- Bug 1 — Cairo context state corruption: The original _draw_grid() changed cr.set_line_width() without restoring it, which would have caused blocks and connections to render with half the expected stroke thickness. Fixed with cr.save()/cr.restore().
- Bug 2 — Only active tab redrawn: The original handler called queue_draw() only on the current page. Toggling the grid would show inconsistent state across tabs. Fixed by iterating main.get_pages() to redraw all open tabs.

Testing: 15 test cases across 4 classes in grc/tests/test_grid_display.py:

Test Class	What It Verifies
TestGridDisplayAction	Action registration, label, tooltip, preference name, default state, toggle behaviour, independence from snap-to-grid
TestGridColor	Color exists, is a valid RGBA 4-tuple, all components in [0, 1], distinct from background
TestGridConstant	CANVAS_GRID_SIZE exists in Constants and is a positive integer
TestDrawingAreaGridMethod	_draw_grid method exists, is callable, has correct signature (self, cr, width, height)

3.3 Issue #8173 / PR #8232 — GRC Crashes on Startup Without bokehgui

PR	#8232 — grc: guard against missing bokeh_gui key in hide_bokeh_gui_options_if_not_installed
Field	**Detail**
Issue	#8173 — GRC Fails on startup when looking for bokeh_gui key in options
Branch	fix/8173-hide-bokeh-gui-no-crash
Commit	915cd04
Author	Aryanth29
Status	PR Open — under maintainer review
Files changed	1 modified + 1 new test file
Issue Link	https://github.com/gnuradio/gnuradio/issues/8173
PR Link	https://github.com/gnuradio/gnuradio/pull/8232

Problem: When GNU Radio Companion starts on an environment where the optional bokehgui dependency is not installed, GRC crashes immediately with an unhandled ValueError: 'bokeh_gui' is not in list. The crash occurs in hide_bokeh_gui_options_if_not_installed(), which calls param['options'].index('bokeh_gui') without checking whether 'bokeh_gui' exists in the list. Since options.block.yml defines options: [] (populated dynamically by workflows at runtime), the element is often absent.

Why This Matters:

- Breaks core GRC startup: Any user without bokehgui installed cannot launch GRC at all.
- Optional feature regression: bokehgui is an optional workflow output generator. Missing optional dependencies should degrade gracefully, never crash the primary application.
- Defensive API design: Hardcoded assumptions that specific options exist in parameter lists create fragility as block specifications evolve.

Solution: A membership check (if 'bokeh_gui' in param['options']:) before calling .index():

- If 'bokeh_gui' is present in param['options'], it is removed alongside its corresponding label — exactly the same behaviour as before.
- If 'bokeh_gui' is absent (or param['options'] is empty), the function executes cleanly without error.

Testing: 3 unit tests in grc/tests/test_hide_bokeh_gui.py:

Test Case	What It Verifies
Bokeh absent from options list	Function does not crash when generate_options contains other options (e.g. ['qt_gui', 'no_gui']) but not 'bokeh_gui'
Empty options list	Function handles empty lists (options: [], option_labels: []) cleanly
Bokeh present in options list	When 'bokeh_gui' is present, both the option key and its label are properly deleted

All 3 tests passed (pytest, 0.03s). The fix adheres to GNU Radio's CONTRIBUTING.md and GREP-0001: DCO sign-off present, single squashed commit, component-prefixed subject line, and unit tests included.

Compliance:

Requirement	Status
DCO Signed-off-by on commit	Present
Single squashed commit	Yes
Subject line: component: prefix	grc: guard against missing bokeh_gui key…
`Commit body lines ≤ 72 chars`	All pass
Unit tests included	3 test cases

3.4 Issue #3110 / PR #8230 — Remove Unused Intensity-Level Markers from Vector Sink

Field	Detail
Issue	#3110 — Two horizontal intensity-level markers in QT GUI Vector Sink are unused
PR	#8230 — qtgui: Remove unused intensity level markers from VectorDisplayPlot
Branch	RishikaBothra:fix-3110-remove-vector-sink-markers
Commit	c6a4976
Status	PR Open — under maintainer review

Problem: Issue #3110 reported that the QT GUI Vector Sink block draws two horizontal intensity-level markers on its plot but these markers serve no purpose. As in the Waterfall Sink where intensity markers let users set the colour-map range the Vector Sink has no controls to move or configure them. The markers are visual clutter left over from copy-pasted plotting code.

Technical Approach:

- Root-cause analysis: VectorDisplayPlot.cc created two QwtPlotMarker objects (d_min_xaxis_marker and d_max_xaxis_marker) plus associated pen, label, and alignment code.
- Fix strategy: Pure dead-code removal. Every line that creates, styles, positions, or references the two unused markers was deleted. No new code was written; the fix is 0 insertions, 112 deletions.
- Files changed (6 files): VectorDisplayPlot.h where removed member declarations, VectorDisplayPlot.cc where removed marker construction and setIntensityRange() method, and four QSS stylesheets where alt.qss, dark.qss, nsf.qss, projector.qss removed unused #VectorDisplayPlot style rules.
- Build verification: cmake --build ~/gnuradio/build --target gnuradio-qtgui completed with no errors or warnings.

Compliance:

Requirement	**Status**
DCO Signed-off-by on commit	Present
Single squashed commit	Yes
Subject line: component: prefix	qtgui: Remove unused intensity level markers from VectorDisplayPlot
Build verified	cmake --build passed (gnuradio-qtgui target)
Headless test	Passed (1 QwtPlotMarker remains; 2 intensity markers removed)

3.5 Contribution Summary Table

Member	**Issue / PR**	**Files Changed**	**Tests / CI**	**Status**
Kensi Patel	#7697 / PR #8224	qa_tcp_sink.py	3 pass, 2 Windows skips; DCO passed	PR Open — under review
Nikkesh Parekh	#5672	6 files (5 modified + 1 new)	15 test cases, 4 classes	Code complete — PR ready
Aryan Thakkar	#8173 / PR #8232	2 files (1 modified + 1 new)	3 tests passed (pytest)	PR Open — under review
Maitri Lad	GSoC analysis, report, testing support	—	—	Documentation & analysis
Rishika Bothra	#3110 / PR #8230	6 files (2 source + 4 QSS)	Build passed; headless test passed	PR Open — under review

4. Proposed System and Methodology

Based on our M2 engineering experience with upstream GNU Radio contributions. The proposed system now consists of three engineering components: a custom GRC block, a Spectrum Sensing block with its framework, and an ISAC (Integrated Sensing and Communication) end-to-end framework rebuilt inside GNU Radio.

4.1 GRC Block Development

We will create a new block in GNU Radio Companion (GRC) and integrate it into the existing GNU Radio environment.

- Study the anatomy of a GRC block: the YAML block definition of parameters, ports, documentation paired with its Python or C++ implementation.
- Implement a new custom block following GNU Radio's module conventions of gr_modtool scaffolding, build system integration via CMake.
- Implement the block so it appears in the GRC block tree and verify it can be dropped into a flowgraph and connected to it like any block.
- Build and test the block end-to-end inside the existing GNU Radio environment to confirm the correctness.

4.2 Spectrum Sensing Block

We will create and add a Spectrum Sensing block with its signal-processing framework implemented.

- Designing the spectrum-sensing framework just like energy detection / FFT-based sensing over incoming IQ samples.
- Implementing the block with configurable parameters such as sample rate, FFT size, and detection threshold.
- Integrate the block into a GRC flowgraph and validate its output against known test signals.

4.3 ISAC End-to-End Framework

Rebuild the ISAC also known as Integrated Sensing and Communication end-to-end framework inside GNU Radio, ensuring the transceiver works perfectly.

- Reconstruct the ISAC transmit and receive chain as GNU Radio blocks/flowgraphs.
- Verify the transceiver works end-to-end: signal generation, transmission, reception, and processing all function together in one flowgraph.
- Validate that sensing and communication functions can operate together within the rebuilt framework.

5. State-of-the-Art Positioning

We position our project at the intersection of practical open-source GNU Radio tooling and academic research on open-set modulation recognition. The table below compares existing approaches against our proposed direction.

Approach	What It Establishes	Gap Our Project Can Investigate
Conventional energy detection	Determines whether signal energy exceeds a threshold	Does not identify signal type or reliably identify unknown classes
Closed-set modulation classifier (e.g., gr-phasma, TorchSig)	Classifies among trained classes with high accuracy	May force unseen signals into a known class no rejection capability
RF ML toolkit (e.g., TorchSig / Sig53)	Provides RF data generation, augmentation, and ML training tooling	Our contribution must be a specific capability (open-set rejection), not simply using a toolkit
GNU Radio classification blocks (e.g., specDetect)	Demonstrate classification in a flowgraph context	Investigate explicit unknown rejection and robust evaluation methodology
Academic AMOSR methods (CFSR, DC-LSTM, GAN-based)	State-of-the-art open-set accuracy in research papers	Complex, computationally expensive, not available as GNU Radio blocks
Our proposed system	Detection + classification + confidence-threshold unknown rejection in a GNU Radio block	Must be demonstrated experimentally before claiming an improvement

5.1 Comparison with Existing GNU Radio Tools

Tool	What It Does	Method	Open-Set?
gr-phasma	Modulation classification OOT module for GNU Radio	Blocks for detection/classification using signal processing & ML	No, closed-set only
TorchSig / Sig53	RF ML framework with GNU Radio block (specDetect) and Sig53 dataset	CNN / transformer / YOLO models trained on Sig53	No, fixed 53-class label set
RadioML dataset	Public dataset of simulated modulated RF signals	Simulated IQ data with various modulations and SNR values	N/A, dataset, not a classifier

Tool	What It Does	Method	Open-Set?
gr-inspector	Signal-analysis tool for detection and separation in wideband spectrum	Energy-based signal detection and feature extraction	Detects unknown signals but does not classify or reject at modulation level
Our proposed block	Classification + open-set rejection in a GNU Radio flowgraph	CNN + confidence-threshold rejection	Yes, flags low-confidence signals as Unknown

6. Results and Evidence

This section presents evidence from our M2 work. Our results at this stage are primarily from open-source contributions (code, tests, CI feedback) rather than from the AI classifier, which is planned for M3.

6.1 PR #8224 — tcp_sink QA (Submitted)

- PR URL: https://github.com/gnuradio/gnuradio/pull/8224

6.2 Issue #5672 — GRC Canvas Grid (Code Complete)

- Solution implemented.

6.3 PR #8232 — bokehgui Crash Fix (Submitted)

- PR URL: https://github.com/gnuradio/gnuradio/pull/8232

6.4 PR #8230 — Vector Sink Markers (Submitted)

- PR URL: https://github.com/gnuradio/gnuradio/pull/8230

6.5 Classifier Results

No measured classifier results are available at this stage.

7. Limitations, Contribution Breakdown and Next Steps

7.1 What Is Complete

- Analysis of two accepted GSoC 2026 proposals (HIL CI and CyberEther).
- PR #8224 submitted to upstream GNU Radio
- Issue #5672 fix implemented
- PR #8232 submitted to upstream GNU Radio
- SOTA positioning updated with comparison table against existing tools and research.
- PR #8230 submitted to upstream GNU Radio

7.2 What Is Partial or In Progress

- PR for Issue #5672 is code-complete but not yet submitted to upstream. The patch and PR description are prepared.

7.3 What Is Not Yet Started

- CNN baseline classifier training on RadioML dataset.
- Confidence-threshold rejection mechanism implementation.
- GNU Radio OOT block (gr-amc-openset) development.
- Colour-coded QT GUI overlay for known/unknown signal display.
- Experimental comparison against naive closed-set baseline and energy-threshold detector.

7.4 Individual Contribution Breakdown

Member	Contributions
Kensi Patel (AU2440102)	PR #8224 https://github.com/gnuradio/gnuradio/pull/8224. Contributed to the report and documentation.
Maitri Lad (AU2440146)	Proposal study.Contributed to the report and documentation.
Rishika Bothra (AU2440179)	PR #8230 https://github.com/gnuradio/gnuradio/pull/8230 Contributed to the report and documentation.
Nikkesh Parekh (AU2440214)	Issue #5672 solution implemented, PR submission pending. Contributed to the project video.
Aryan Thakkar (AU2440248)	PR #8232 https://github.com/gnuradio/gnuradio/pull/8232 Contributed to the project video.

7.5 Next Steps (M3 Engineering Milestones)

- Submit PR for Issue #5672 to upstream GNU Radio.
- Generate a subset of RadioML data and train a baseline CNN classifier on selected modulation types.
- Implement the confidence-threshold rejection mechanism as a layer above the classifier.
- Develop the GNU Radio OOT block (gr-amc-openset) and integrate into a GRC flowgraph.
- Develop the colour-coded QT GUI overlay (green = known, red = unknown).
- Compare the open-set system against naive closed-set classification and energy-threshold detection.
- Address maintainer review feedback on PRs #8224, #8230 and #8232 and complete merge.

- Upload M2_G5/ submission folder to the public course repository.

8. References

[1] GNU Radio. (n.d.). GNU Radio — The free and open software radio ecosystem. GitHub. https://github.com/gnuradio/gnuradio

[2] GNU Radio Foundation. (n.d.). Google Summer of Code — GNU Radio. https://www.gnuradio.org/gsoc/

[3] Maity, S. (2026). Hardware-in-the-Loop CI for GNU Radio — GSoC 2026 Final Report. Google Summer of Code.

[4] Mahmoud, Y. (2026). gr-cyberether: GPU-accelerated visualization sinks for GNU Radio — GSoC 2026. GitHub. https://github.com/luigicruz/gr-cyberether

[5] Mueller, M. (2025). tcp_source has no QA [GitHub Issue #7697]. GNU Radio. https://github.com/gnuradio/gnuradio/issues/7697

[6] KensiPatel. (2026). network: add tcp_sink QA coverage [Pull Request #8224]. GNU Radio. https://github.com/gnuradio/gnuradio/pull/8224

[7] Kozel, D. (2020). Add option to display grid in GRC canvas [GitHub Issue #5672]. GNU Radio. https://github.com/gnuradio/gnuradio/issues/5672

[8] Issue #8173 — GRC Fails on startup when looking for bokeh_gui key in options. GNU Radio. https://github.com/gnuradio/gnuradio/issues/8173

[9] Aryanth29. (2026). grc: guard against missing bokeh_gui key in hide_bokeh_gui_options_if_not_installed [Pull Request #8232]. GNU Radio. https://github.com/gnuradio/gnuradio/pull/8232

[10] Vallance, P., Oh, E., Mullins, J., Gulati, M., Hoffman, J., & Carrick, M. (2024). TorchSig: A GNU Radio block and new spectrogram tools for augmenting ML training. Proceedings of the GNU Radio Conference. https://pubs.gnuradio.org/index.php/grcon/article/view/147

[11] Triantafyllakis, C. (n.d.). gr-phasma: A GNU Radio out-of-tree module for automatic modulation classification experimentation. GitHub. https://github.com/ctriant/gr-phasma

[12] O'Shea, T., & West, N. (2016). Radio machine learning dataset generation with GNU Radio. Proceedings of the GNU Radio Conference (GRCon16). https://pubs.gnuradio.org/index.php/grcon/article/download/11/10

[13] Group 05 Project Repository. https://github.com/Aryanth29/g5_gsoc_spectrumsensing

[14] Issue #3110 — Two horizontal intensity level markers in qtgui Vector Sink are unused. GNU Radio. https://github.com/gnuradio/gnuradio/issues/3110

[15] RishikaBothra. (2026). qtgui: Remove unused intensity level markers from VectorDisplayPlot [Pull Request #8230]. GNU Radio. https://github.com/gnuradio/gnuradio/pull/8230